CSEN 11 Advanced Programming

Strings – the null terminated character arrays

Character

- **What is a char in C?**
  - A char is **1 byte** (8 bits)
  - That byte stores a **number**
  - We *interpret* that number using **ASCII**

```
char c = 'A';
```

- **What's actually stored?**
  - ASCII of 'A' = **65**

https://www.asciitable.com

  - Memory holds **65**, not the letter A

Ascii Table

C does not know letters.
It only knows numbers.

```
printf("%d\n", 'A');   // 65
printf("%c\n", 65);    // A
```

https://www.asciitable.com

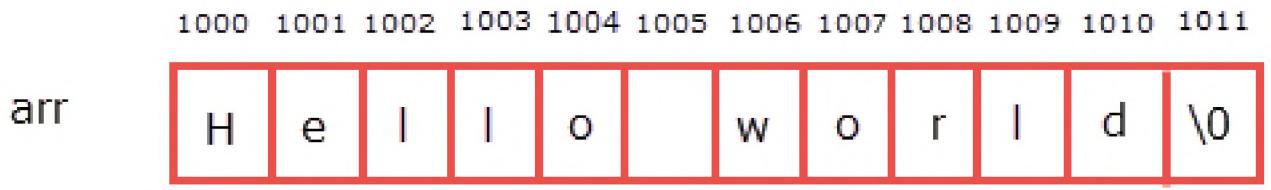
Dec	Hx	Oct	Char		Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL	(null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH	(start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX	(start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX	(end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT	(end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ	(enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK	(acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL	(bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS	(backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB	(horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF	(NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT	(vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF	(NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR	(carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO	(shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI	(shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE	(data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1	(device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2	(device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3	(device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4	(device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK	(negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN	(synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB	(end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN	(cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM	(end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB	(substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC	(escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS	(file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS	(group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS	(record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US	(unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Character Array

- A <u>character array</u> is an array in which the individual elements are stored as characters
- Each Character requires 1 byte of storage.

Character String

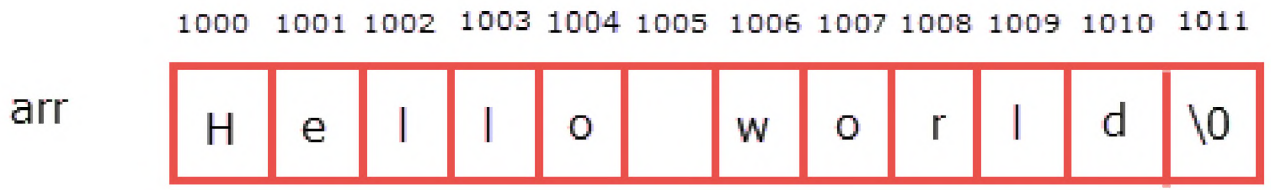
- A <u>character string</u> is a character array in which the last element is a NULL character ‘\0’, which has an ASCII integer equivalent to zero.
- **Every C string needs one extra byte for '\0'.**



12 bytes of memory is allocated to store 12 characters

Character String

- A <u>character string</u> is a character array in which the last element is a NULL character ‘\0’, which has an ASCII integer equivalent to zero.
- **Every C string needs one extra byte for '\0'.**



12 bytes of memory is allocated to store 12 characters

String Variable Declaration

- Array of characters:

```
char s[10];
```

  - Declares a c-string variable to hold up to 9 characters plus one null character
  - No initial value

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
?	?	?	?	?	?	?	?	?	?

    - If s contains string “Hi Mom!”, then stored as:

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
H	i		M	o	m	!	\0	?	?

String Variable Initialization

Can initialize string:

```
char message[15] = "Hi There";
```

- Need not fill entire array
- Initialization places '\0' at end

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
H	i		T	h	e	r	e	\0	?	?	?	?	?	?

THIS IS NOT A STRING. CHAR ARRAY.

```
char abc[] = {'a', 'b', 'c'};
```

[0]	[1]	[2]
a	b	c

- Can omit array-size:

```
char abc[] = "abc";
```

[0]	[1]	[2]	[3]
a	b	c	\0

- Automatically makes size one more than length of quoted string

```
char abc[] = {'a', 'b', 'c', '\0'};
```

Strings are Arrays.

```
char abc[] = {'a', 'b', 'c', '\0'};
```

[0]	[1]	[2]	[3]
a	b	c	\0

- Can access indexed variables of:

```
char hi[5] = "Hi";
```

  - hi[0] is ‘H’
  - hi[1] is ‘i’
  - hi[2] is ‘\0’
  - hi[3] is unknown
  - hi[4] is unknown

[0]	[1]	[2]	[3]	[4]
H	i	\0	?	?

- Can manipulate array elements

```
char dobedo[7] = "DoBeDe";
dobedo[5] = 'o';
dobedo[6] = '!';
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	
D	o	B	e	D	e	\0	?
D	o	B	e	D	o	\0	?
D	o	B	e	D	o	!	?

- Be careful!
- Here, ‘\0’ (null) was overwritten by a ‘!’

  - If null overwritten, string no longer ‘acts’ like a string!

String Output

- Use printf
- Compiler knows the stop printing when it sees a ‘\0’

```
char string[100];
...
printf (“String: %s\n”, string);
```

```
// DO NOT print out char arrays as strings
// It expects '\0’. Will run past memory.

char a[5] = {'H','e','l','l','o'};
printf("%s", a);   // undefined behavior
```

Arrays of Strings

- A string is an array of characters
- An array of strings is a 2D array of characters
- Example, a list of 10 names, each with at most 19 characters:

```
char    names[10][20];
```

String Functions

- #include <string.h>

strlen (s) – lenght of the string s (without the \0)

strcpy (s, t) – copy t to s

strncpy (s, t, n) – copy n characters from t to s

strcat (s, t) – concatenates t to the end of s

strncat (s, t, n) – concatenates n characters from t to the end of s

strcmp (s, t) – compares s and t (<: -1, ==: 0, >: 1)

strncmp (s, t, n) – compares at most n characters of s to t

Strings Manipulations

```
#include <string.h>
```

1. Strings Length

- Often useful to know length of string

  `strlen(string)`

  - Returns number of characters
    - Does not include null
    - Return type is size_t so type cast may be required

```
char hello_world[] = "Hello World";
printf("%d", (int) strlen(hello_world));
// The above will print number 11
```

NOTE: printf("%lu",sizeof(hello_world)); will print 12. this includes the \0

2. String copy

- Strings are not like other variables, they are arrays
  - Cannot assign:

```
char msg[10];
msg = "Hello";     // ILLEGAL!
```

- Must use string library function for assignment:

  `strcpy(destination, source)`

  - NO checks for size – up to programmer!
  - 'Assign' value of msg to "Hello":

```
strcpy(msg, "Hello");
```

- Or **`strncpy(destination, source, limit)`**
  - No ending null character if limit is reached

Key assumptions:
- dest already has **enough space**
- src is a **valid C string**

C does **not** check either

strcpy completely overwrites the contents of dest.

3. String Concatenate

- Appends one string onto end of another

  `strcat(destination, source)`

```
char msg1[30] = "Hello";
char msg2[30] = "Hello";
strcat(msg1, "World");  // Result "HelloWorld"
strcat(msg2, " World"); // Result "Hello World"
```

  - Be careful when concatenating words
    - msg1 is missing space after Hello
    - msg2 is correct

4. String Compare

- Cannot use operator == to compare

```
char hello[] = "Hello";
char goodbye[] = "Goodbye";
if (hello == goodbye)  // NOT ALLOWED
```

- Must use strcmp string library function to compare:

  `strcmp(string1, string2)`

  - Returns zero int if string1 is equal to string 2
  - Returns <0 int if string1 is less than string2
  - Returns >0 int if string1 is greater than string2

```
if (strcmp(hello, goodbye) == 0)
   printf("Hello equal to Goodbye");
else if (strcmp(hello, goodbye) < 0)
   printf("Hello less than Goodbye");
else
   printf("Hello greater than Goodbye");
```

4. String Compare

strcmp compares strings lexicographically (dictionary order) using ASCII values.
It compares **character by character**, from left to right.

```
strcmp("cat", "dog")
'c' (99) vs 'd' (100)
99 - 100 = -1  → negative
```

```
strcmp("cat", "cater")
c = c
a = a
t = t
'\0' (0) vs 'e' (101)
0 - 101 = negative
```

```
strcmp("cater", "cat")
c = c
a = a
t = t
'e' (101) vs '\0' (0)
101 - 0 = positive
```

```
strcmp("Zoo", "apple")
'Z' (90) vs 'a' (97)
90 - 97 = negative
So "Zoo" < "apple" (ASCII order!)
```

Strings in Functions

- A string is an array, so
  - String parameter is an array parameter
  - Strings passed to a function can be changed by the receiving function!
- Like all arrays, typical to send size as well
  - Function could also use '\0' to find end

```
char msg[] = "BOB";
int msg_len = strlen(msg);
str_reverse(msg, msg_len);
```

String Conversions

- String to Number
  - Function atoi converts a string to an integer

```
int  x;
char     str[ ] = “123”;
…
x = atoi (str);
```

→Also, strtoint and strtodouble

String Conversions

- Number to String
  - Function sprintf converts a number to a string

```
int  x = 123;
float     y = 45.46;
char      str[20];
...
sprintf (str, “%d %f”, x, y);
```